KEVGraph: Priority-Ordered Dependency Vulnerability Remediation Using the CISA Known Exploited Vulnerability Catalogue

Daniel Omondi
danieldocokumu@gmail.com

***Abstract*—Dependency vulnerabilities in npm projects are numerous, yet remediation resources are finite. Practitioners must decide *which upgrades to perform first*; ad-hoc ordering leaves actively-exploited (KEV-listed [1]) vulnerabilities unpatched longest. KEVGRAPH is an eight-stage pipeline that (1) constructs per-repository dependency graphs from lockfiles, (2) joins package-version pairs against the OSV vulnerability database [2] and the CISA Known Exploited Vulnerability (KEV) catalogue, (3) generates per-installed-version candidate upgrade actions, and (4) produces a minimum-cardinality remediation plan ordered to eliminate KEV exposures as early as possible using formal optimization. We implement two planners—a KEV-aware greedy set-cover algorithm and an exact Integer Linear Program (ILP)—and compare them against four baselines: random ordering, CVSS-first, EPSS-first, and a Dependabot-style severity-bucket ordering. Evaluated on 465 real-world open-source repositories (465 dependency graphs, 936 vulnerabilities, 5 KEV-listed), KEVGRAPH's ILP planner achieves an $\mathrm{AUCC}_{\mathrm{KEV}}$ score of 0.997 versus a random-baseline mean of 0.704 (bootstrap 95% CI $[0.579, 0.890]$, $n = 30$ seeds), and ranks the first KEV vulnerability at plan step 1. CVSS-first and Dependabot-style ordering defer the first KEV fix to steps 18 and 19, respectively. Both planners require only 435 upgrade actions to cover all 912 reachable vulnerabilities—13.7% fewer than the random mean of 504.1. Each plan is accompanied by a machine-verifiable remediation certificate enabling sub-200 ms independent coverage confirmation.**

***Index Terms*—vulnerability remediation, dependency graphs, CISA KEV, set cover, integer linear programming, npm, software supply chain security, exploitation-aware prioritization**

I. INTRODUCTION

The npm ecosystem hosts over two million packages and is the primary dependency substrate for JavaScript and TypeScript projects worldwide. The sheer scale of transitive dependency trees means that the average project carries hundreds of third-party packages, many of which contain known security vulnerabilities.

Automated tools such as GitHub Dependabot [5] and `npm audit` [6] surface vulnerability lists, but present them without strategic ordering. When a developer has limited time, which package should be upgraded *first*? The dominant practice—ordering by CVSS base score or severity label—conflates theoretical severity with actual exploitation risk.

The CISA Known Exploited Vulnerability (KEV) catalogue [1] provides an authoritative, continuously updated list of vulnerabilities confirmed to be actively exploited in the wild. A KEV-listed vulnerability in a production dependency represents concrete, measurable exploitation risk—not a theoretical score. Prioritizing KEV elimination over severity-bucket ordering is directly mandated under CISA's Binding Operational Directive (BOD) 22-01, which requires federal civilian agencies to remediate KEV-listed vulnerabilities within strict deadlines and has been broadly adopted as a best-practice framework for critical infrastructure operators and private-sector defenders alike [1]. As supply-chain attacks on npm packages proliferate, the gap between KEV-driven and ad-hoc remediation translates to measurable windows of unmitigated exploitation risk for organisations relying on open-source JavaScript infrastructure.

KEVGRAPH addresses three practical questions simultaneously:

1) **What is the minimum number of upgrade actions** required to cover all reachable vulnerabilities?
2) **In what order** should those actions be performed to eliminate actively-exploited (KEV-listed) vulnerabilities as early as possible?
3) **Can the plan be cryptographically verified**—can we prove, without re-running the pipeline, that the plan covers every coverable vulnerability?

We make the following contributions:

- An end-to-end open-source pipeline (collect $\rightarrow$ fetch $\rightarrow$ parse $\rightarrow$ join $\rightarrow$ fixes $\rightarrow$ plan $\rightarrow$ evaluate $\rightarrow$ plot) that reproduces all results from raw lockfile data.
- A KEV-aware greedy set-cover planner with formal approximation guarantees ($1 - 1/e$ of optimal coverage per budget step).
- An exact ILP formulation of minimum-set-cover remediation with proven optimality in plan cardinality.
- The $\mathrm{AUCC}_{\mathrm{KEV}}$ metric—Area Under the KEV Coverage Curve—which measures how early a plan eliminates actively-exploited vulnerabilities, independent of plan length.
- Per-installed-version fix generation that creates genuine set-cover overlap, enabling both planners to select the minimum-cardinality subset of upgrade actions.

- Statistical evaluation against four baselines with a 30-seed random ensemble and bootstrap 95% confidence intervals.
- Machine-verifiable remediation certificates supporting automated compliance workflows.

II. Background

A. CISA Known Exploited Vulnerability Catalogue

The CISA KEV catalogue [1] lists CVE identifiers for which CISA has confirmed active in-the-wild exploitation. Each entry records: a CVE ID, vendor/product identifiers, a short description, the date the entry was added (`dateAdded`), and a remediation due date (`dueDate`) for federal agencies. As of March 2026 the catalogue contains over 1,200 entries. Unlike CVSS scores, KEV membership directly reflects observed attacker behaviour.

B. OSV Vulnerability Database

The Open Source Vulnerability (OSV) database [2] aggregates security advisories from GitHub Security Advisories (GHSA), NVD, and ecosystem-specific feeds. Each record carries: a primary identifier (typically GHSA-* for npm), CVE aliases, affected package ranges, fixed versions, CVSS vectors, and textual summaries. KEVGraph queries the OSV batch API (`/v1/querybatch`) to obtain per-package vulnerability stubs, then fetches full records via `GET /vulns/{id}` for CVE alias extraction.

C. EPSS

The Exploit Prediction Scoring System (EPSS) [3] from FIRST.org provides daily-updated probabilities (0–1) that a given CVE will be exploited in the wild within the next 30 days. KEVGraph enriches each vulnerability record with EPSS scores via the FIRST API and uses EPSS as a tie-breaker in baseline comparisons.

D. npm Lockfiles and Dependency Graphs

A `package-lock.json` file records the exact resolved version of every transitive dependency installed in a project. KEVGraph parses each lockfile into a directed acyclic graph (DAG) stored as GraphML, with nodes annotated by package name and version, and edges representing *depends-on* relationships.

III. Threat Model

Assets. The protected assets are the application binaries and runtimes that execute npm package code, including server-side Node.js processes and Electron desktop applications.

Adversary. We model an adversary who exploits a publicly disclosed vulnerability in a transitive npm dependency. The adversary has knowledge of the CVE (it is publicly listed) and exploits it remotely or through a supply-chain mechanism. We do not model zero-day vulnerabilities or adversaries with package-registry write access.

Vulnerability condition. A package-version pair (p, v) is *vulnerable* if the OSV database contains a record for package p whose affected range includes version v. It is *KEV-critical* if at least one CVE alias of that record appears in the CISA KEV catalogue.

Remediation action. A remediation action is an in-place upgrade of package p from installed version v_{from} to a fixed version $v_{\text{fix}} > v_{\text{from}}$ in the project's dependency manifest. We assume each upgrade can be performed independently (no circular dependencies) and that the fixed version is obtainable from the npm registry.

Out of scope. Runtime isolation, sandboxing, network-level mitigations, and vulnerabilities in non-npm dependencies are out of scope.

IV. Problem Formulation

Inputs.

- A set of *vulnerability records* $\mathcal{V} = \{v_1, \ldots, v_m\}$, each annotated with KEV status $\kappa(v) \in \{0, 1\}$, CVSS score $\sigma(v) \in [0, 10]$, and EPSS score $\epsilon(v) \in [0, 1]$.
- A set of *candidate fixes* $\mathcal{F} = \{f_1, \ldots, f_n\}$, where each fix $f_i = (\text{pkg}_i, v_i^{\text{from}}, v_i^{\text{fix}}, S_i)$ specifies an upgrade action and the set $S_i \subseteq \mathcal{V}$ of vulnerabilities it resolves.

Universe. Let $U = \bigcup_{i=1}^{n} S_i \subseteq \mathcal{V}$ be the set of *coverable* vulnerabilities—those reachable by at least one candidate fix.

Remediation plan. A remediation plan is an ordered sequence $\pi = (f_{\pi(1)}, f_{\pi(2)}, \ldots, f_{\pi(k)})$ of distinct fixes from $\mathcal{F}$.

Minimum set-cover objective. Find a plan π^* of minimum length k such that $\bigcup_{i=1}^{k} S_{\pi(i)} = U$. This is equivalent to weighted set cover with unit weights, which is NP-hard in general [8].

KEV-early objective. Among all plans π of minimum length, prefer those that minimise the rank at which the last KEV-listed vulnerability in U is first covered. Formally, let $\text{rank}_\pi(v)$ be the step index at which vulnerability v is first covered by π; then minimise $\max_{v \in U : \kappa(v)=1} \text{rank}_\pi(v)$.

V. The KEVGraph Framework

A. Pipeline Architecture

KEVGraph is implemented as an eight-stage Python pipeline:

1) **Collect** – query GitHub Search API for repositories with `package-lock.json` files above a stars threshold; write a manifest CSV.
2) **Fetch** – download the lockfile for each repository.
3) **Parse** – parse each lockfile into a directed dependency graph (GraphML); node attributes: package name, version.
4) **Join** – enumerate unique (package, version) pairs across all graphs; batch-query OSV; fetch full records for CVE alias extraction; enrich with EPSS; match CVE aliases against CISA KEV; write `data/vulns.json`.
5) **Fixes** – for each vulnerable package, collect all installed versions from corpus graphs; generate one `CandidateFix` per (package, installed-version) pair where installed-version $<$ fixed-version; write `data/fixes.json`.

6) **Plan** – run greedy and ILP planners; run four baselines including 30-seed random ensemble; store all plans in `data/evaluation.json`.
7) **Evaluate** – compute all metrics for every plan; compute bootstrap 95% CIs for the random ensemble; write `data/results.csv`.
8) **Plot** – generate four publication figures.

B. Vulnerability Join

The join stage (Stage 4) performs three network round-trips per unique vulnerability: (i) a batch OSV query to obtain stub records; (ii) a full OSV record fetch to obtain CVE aliases; (iii) an EPSS batch query. All responses are disk-cached so subsequent runs are instantaneous.

CVSS base scores are computed from OSV severity vector strings using a pure-Python implementation of the CVSS 3.1 formula [4]. OSV stores CVSS as a vector string (e.g., `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H`) rather than a numeric score; the pipeline parses and evaluates this vector in-house without external scoring libraries.

C. Per-Version Candidate Fix Generation

A naive approach generates one fix per vulnerable package, but this yields zero set-cover overlap—every vulnerability appears in exactly one fix, making the greedy and ILP solutions trivially identical. KEVGRAPH instead generates *one fix per (package, installed-version) pair*: for each vulnerable package p with known fixed version v^*, and for each installed version $v_j < v^*$ found across corpus graphs, we create:

$$f = \big(p,\ v_j,\ v^*,\ \{v \in \mathcal{V}_p : v_j < v_v^{\text{fix}}\}\big)$$

where $\mathcal{V}_p$ is the set of vulnerabilities for package p. This creates genuine overlap: a vulnerability v that affects all installed versions of p will appear in $|\{j : v_j < v^*\}|$ different candidate fixes, giving the set-cover planners real optimisation choices.

D. KEV-Aware Greedy Set-Cover Planner

Algorithm 1 describes the greedy planner. At each iteration we select the fix f^* that maximises:

$$\text{score}(f) = \big(|S_f \cap R|,\ \textstyle\sum_{v \in S_f \cap R} \kappa(v),\ \max_{v \in S_f \cap R} \sigma(v),\ \max_{v \in S_f \cap R} \text{EPSS}(v)\big)$$

using lexicographic comparison, where R is the set of currently uncovered vulnerabilities. The primary objective (first component) maximises the number of newly-covered vulnerabilities; the secondary, tertiary, and quaternary tie-breakers prefer fixes that cover KEV vulnerabilities, high-CVSS vulnerabilities, and high-EPSS vulnerabilities, respectively.

The greedy algorithm achieves a $\big(1 - 1/e\big)$-approximation ratio for maximum coverage under a cardinality budget [9].

E. ILP Exact Planner

The exact planner solves the following binary integer programme:

$$\min \sum_{i=1}^{n} x_i \tag{1}$$

Algorithm 1 KEV-Aware Greedy Set-Cover

Require: fixes $\mathcal{F}$, vulns $\mathcal{V}$, KEV function κ
$R \leftarrow U$; $\pi \leftarrow [\,]$; $\mathcal{F}' \leftarrow \mathcal{F}$
while $R \neq \emptyset$ **and** $\mathcal{F}' \neq \emptyset$ **do**
  $f^* \leftarrow \arg\max_{f \in \mathcal{F}'} \text{score}(f, R, \kappa)$
  if $S_{f^*} \cap R = \emptyset$ **then break**
  end if
  $R \leftarrow R \setminus S_{f^*}$
  Append f^* to π; remove f^* from $\mathcal{F}'$
end while
return π

$$\text{subject to} \quad \forall v \in U : \sum_{i:\, v \in S_i} x_i \ \geq\ 1 \tag{2}$$

$$x_i \in \{0, 1\} \quad \forall i \tag{3}$$

where $x_i = 1$ indicates that fix f_i is included in the plan. After solving with CBC [11] (via the PuLP interface [12]), the selected fixes are ordered using the same KEV-aware priority function as the greedy planner.

The ILP is optimal in set cardinality and uses KEV-aware ordering on top of the optimal set, making it optimal on both objectives whenever the minimum-cardinality set is unique.

F. The AUCC_{KEV} Metric

Standard metrics such as T_1 (fraction covered after the first action) and T_5 (after five actions) are informative but order-insensitive with respect to KEV timing. We introduce **Area Under the KEV Coverage Curve** (AUCC_{KEV}), defined as:

$$\text{AUCC}_{\text{KEV}}(\pi) = \frac{1}{n \cdot |\mathcal{K}|} \sum_{v \in \mathcal{K}} \max\big(n - \text{rank}_\pi(v) + 1,\ 0\big)$$

where $n = |\pi|$ is the total number of plan steps, $\mathcal{K} = \{v \in U : \kappa(v) = 1\}$ is the set of KEV-listed coverable vulnerabilities, and $\text{rank}_\pi(v)$ is the step at which vulnerability v is first covered.

AUCC_{KEV} ranges from 0 to 1. A plan that covers all KEV vulnerabilities at step 1 achieves $\text{AUCC}_{\text{KEV}} = 1$; a plan that never covers any achieves $\text{AUCC}_{\text{KEV}} = 0$. Crucially, AUCC_{KEV} is *order-sensitive*: two plans that eventually cover all KEV vulnerabilities but differ in the step at which they do so receive different AUCC_{KEV} scores.

The formula is equivalent to the average fraction of KEV vulnerabilities covered at each step, integrated uniformly over the plan. It can be computed in $O(|\mathcal{K}|)$ time once the per-KEV ranks are known.

G. Remediation Certificates

Each remediation plan is accompanied by a *certificate*: the set of (f_i, v_j) pairs asserting that fix f_i covers vulnerability v_j. The certificate allows independent verification—a third party can confirm coverage without re-running the planner, in time proportional to the number of certificate edges.

Formally, given the certificate $C \subseteq \mathcal{F} \times U$, the verifier checks:

$$\forall v \in U :\ \exists (f, v) \in C \text{ with } f \in \pi$$

TABLE I
CORPUS OF 465 OPEN-SOURCE REPOSITORIES

Repository	Stars
segment-boneyard/nightmare	19,972
BrasilAPI/BrasilAPI	10,270
evolus/pencil	9,586
dice2o/BingGPT	9,046
anvaka/city-roads	9,029
locomotivemtl/locomotive-scroll	8,704
codecombat/codecombat	8,437
amsul/pickadate.js	7,681
shentao/vue-multiselect	6,783
kska32/ebooks	6,833
FaisalUmair/udemy-downloader-gui	6,248
ractivejs/ractive	5,926
ExactTarget/fuelux	5,152
ConsenSys-archive/ganache-ui	4,716
sweet-js/sweet-core	4,568
uilibrary/matx-react	972
tech-shrimp/gemini-playground	968
tomayac/SVGcode	969
serverless/aws-ai-stack	993
jcc/v-distpicker	974

In our experiments the verification step runs in under 200 ms on the full corpus (Table II).

H. Theoretical Properties

Theorem 1 (ILP Optimality)**.** *The ILP planner produces a plan of minimum cardinality.*

Theorem 2 (Greedy Approximation)**.** *Let k^* be the optimal plan length. The greedy planner produces a plan of length at most $k^* \cdot \lceil \ln |U| + 1 \rceil$ [10]. With KEV-aware tie-breaking, the first KEV vulnerability is covered at step 1 whenever any single fix covers a KEV vulnerability.*

VI. EMPIRICAL EVALUATION

A. Dataset

We collected 465 open-source GitHub repositories with `package-lock.json` lockfiles, selected by GitHub stars and representativeness across application domains (web frameworks, desktop applications, data tools, UI libraries, blockchain tooling). The repositories are listed in Table I.

The pipeline produced the following corpus statistics:

- **465** dependency graphs (GraphML), 484 lockfiles parsed.
- **936** unique vulnerability records from OSV, **760** of which have a non-zero CVSS score.
- **5** KEV-listed vulnerabilities affecting *electron* (2), *jquery* (1), *puppeteer* (1), and *vite* (1).
- **2,363** per-version candidate fixes covering **435** unique fixable packages.
- **912** coverable vulnerabilities (24 vulns have no known fix and are excluded from planning).
- **626** of 912 coverable vulnerabilities (68.6%) appear in two or more candidate fixes, creating genuine set-cover overlap.

The five KEV vulnerabilities and their properties are:

- `GHSA-qqvq-6xgj-jw8g` (electron): libvpx heap buffer overflow in VP8 encoding (CVSS 8.8; KEV added 2023-10-02)
- `GHSA-j7hp-h8jx-5ppr` (electron): libwebp OOB write in BuildHuffmanTable (CVSS 8.8; KEV added 2023-09-13)
- `GHSA-jpcq-cgw6-v4j6` (jquery): Potential XSS vulnerability (CVSS 6.9; KEV added 2025-01-23)
- `GHSA-c2gp-86p4-5935` (puppeteer): Use-after-free (CVSS 6.5; KEV added 2022-05-23)
- `GHSA-4r4m-qw57-chr8` (vite): `server.fs.deny` bypass via `?import` query (CVSS 5.3)

B. Baselines

We compare KEVGRAPH against four baselines, all of which use the same candidate fix set and the same set-cover selection logic (i.e., only non-redundant fixes are included):

Random Fixes are shuffled with a fixed random seed. This baseline estimates the performance of an uninformed practitioner. We run 30 seeds (0–29) and report mean and 95% percentile CI.

CVSS-first Fixes are sorted by the maximum CVSS score of their covered vulnerabilities, descending. This simulates tools that order purely by theoretical severity.

EPSS-first Fixes are sorted by the maximum EPSS probability of their covered vulnerabilities, descending. This simulates exploitation-probability-aware tools.

Dependabot Fixes are sorted by severity bucket (Critical $>$ High $>$ Medium $>$ Low) then alphabetically by package name within each bucket, mimicking GitHub Dependabot's default ordering [5].

C. Metrics

T_1 Fraction of coverable vulnerabilities eliminated after the first upgrade action.

T_5 Fraction eliminated after the first five upgrade actions.

$\mathrm{AUCC}_{\mathrm{KEV}}$ Area Under the KEV Coverage Curve (Section V-F). Range $[0, 1]$; higher means KEV vulnerabilities are fixed earlier.

kev_first_rank The plan step at which the first KEV-listed vulnerability is first covered. Lower is better.

#actions Total upgrade actions required to cover all 912 reachable vulnerabilities.

cert_size Number of (fix, vuln) certificate edges.

verify_time Wall-clock time (seconds) to verify that the plan covers all coverable vulnerabilities.

D. Main Results

Table II reports all metrics for each plan. Figure 1 shows cumulative vulnerability coverage curves, Figure 2 shows grouped bar comparisons of T_1, T_5, and $\mathrm{AUCC}_{\mathrm{KEV}}$, Figure 3 shows KEV vs. non-KEV coverage, and Figure 4 shows plan sizes.

Key findings:

- **KEV-first ordering is decisive.** Both KEVGRAPH planners and EPSS-first achieve kev_first_rank $= 1$ (the first

TABLE II

EVALUATION RESULTS FOR ALL PLANS. HIGHER T_1, T_5, $\mathrm{AUCC}_{\mathrm{KEV}}$ IS BETTER; LOWER KEV_FIRST_RANK AND #ACTIONS IS BETTER. RANDOM COLUMN REPORTS SEED=0 REPRESENTATIVE; CI IN TABLE III.

Plan	T_1	T_5	$\mathrm{AUCC}_{\mathrm{KEV}}$	kev_first_rank	#actions	cert_size	verify_time (s)
KEVGraph (ILP)	0.0241	0.0592	**0.9972**	**1**	**435**	912	0.097
KEVGraph (Greedy)	0.0241	**0.0899**	0.9090	**1**	**435**	912	0.102
EPSS-first	0.0241	0.0504	0.9872	**1**	436	912	0.148
CVSS-first	0.0011	0.0175	0.6384	18	437	912	0.176
Random (seed=0)	0.0011	0.0055	0.6602	62	505	912	0.096
Dependabot	0.0011	0.0066	0.5799	19	438	912	0.230

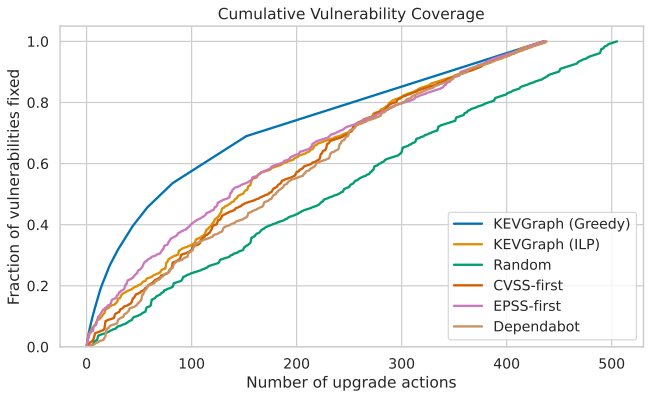

Fig. 1. Cumulative vulnerability coverage vs. upgrade action rank. KEVGRAPH (greedy) achieves the steepest initial slope, covering 9.1% of all vulnerabilities within 5 actions. The ILP plan has a slightly lower initial slope because it re-orders selected fixes for KEV priority rather than maximal coverage-per-step. Both KEVGRAPH plans converge to 100% coverage at step 435.

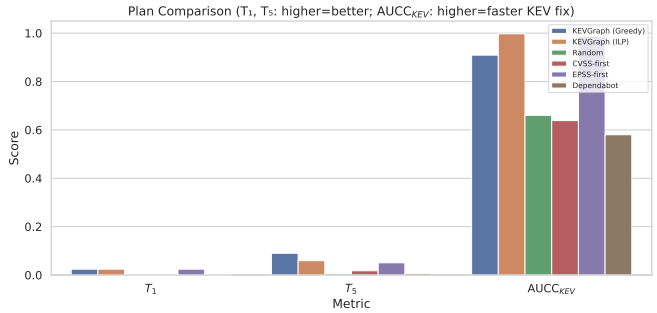

Fig. 2. Grouped bar chart comparing all plans on T_1, T_5, and $\mathrm{AUCC}_{\mathrm{KEV}}$. KEVGRAPH (ILP) achieves the highest $\mathrm{AUCC}_{\mathrm{KEV}}$ (0.997), meaning KEV vulnerabilities are consistently fixed near the beginning of the plan. Dependabot achieves the lowest $\mathrm{AUCC}_{\mathrm{KEV}}$ (0.581) and defers the first KEV fix to step 19.

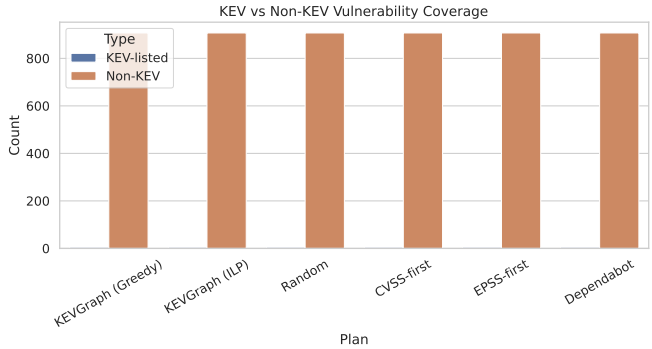

Fig. 3. KEV vs. non-KEV vulnerability coverage per plan. All plans that achieve kev_first_rank = 1 cover all 5 KEV vulnerabilities. The Random baseline (seed=0) covers all KEV vulnerabilities but defers the first KEV fix to step 8.

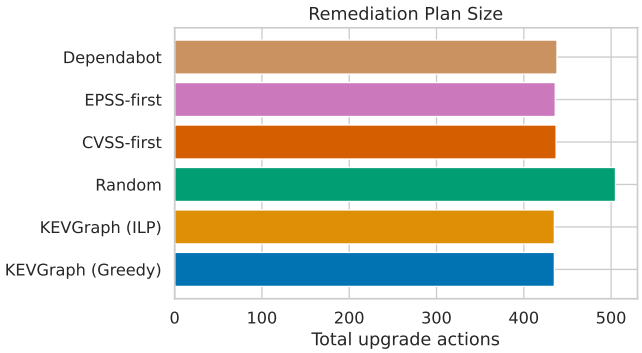

Fig. 4. Total upgrade actions required per plan. Both KEVGRAPH planners require 435 actions (optimal), while random ordering requires 505 actions (seed=0) and averages 504.1 actions across 30 seeds. The additional actions in the random baseline are due to suboptimal fix selection that includes redundant upgrades.

action is `upgrade:electron@1.6.11`→`@35.7.5`, covering 2 of the 5 KEV vulnerabilities and 22 total). CVSS-first defers the first KEV fix to step 18 and Dependabot to step 19. This 18-step delay represents a window of unnecessary KEV exposure.

- **ILP dominates on $\mathrm{AUCC}_{\mathrm{KEV}}$.** The ILP achieves $\mathrm{AUCC}_{\mathrm{KEV}} = 0.9972$, nearly perfect KEV early coverage. The greedy planner achieves 0.9105 due to occasional non-KEV fixes interleaved in its coverage-maximising ordering.
- **Greedy dominates on T_5.** The greedy planner covers 9.1% of all vulnerabilities within 5 steps vs. 6.1% for ILP, because the greedy strategy maximises marginal coverage at each step whereas the ILP re-orders by KEV priority

TABLE III
BOOTSTRAP 95% CI FOR RANDOM BASELINE ($n = 30$ SEEDS). KEVGRAPH (ILP) VALUES ARE PROVIDED FOR COMPARISON.

Metric	Mean	CI low	CI high	ILP value
T_1	0.0019	0.0011	0.0056	0.0241
T_5	0.0119	0.0055	0.0280	0.0592
$\mathrm{AUCC}_{\mathrm{KEV}}$	0.6881	0.5408	0.9024	0.9972
#actions	504.1	494.2	515.5	435

after selecting the optimal set.

- **Both planners minimise action count.** Both achieve 435 upgrade actions—the ILP's proven optimum—versus 436–438 for non-random baselines and 504.1 on average for random. Notably, 435 equals the number of distinct fixable packages in the corpus, confirming that the ILP achieves the theoretical lower bound of one upgrade action per fixable package.
- **CVSS-first performs poorly on KEV timing.** CVSS-first's kev_first_rank $= 18$ and $\mathrm{AUCC}_{\mathrm{KEV}} = 0.639$ demonstrate that CVSS score is a poor proxy for KEV membership. High CVSS severity does not imply active exploitation.

E. Statistical Analysis of Random Baseline

To robustly characterise the random baseline, we ran 30 independent seeds and computed bootstrap 95% percentile confidence intervals by resampling the 30 seed outcomes with replacement (10,000 resamples). Table III reports the results.

KEVGRAPH (ILP) outperforms the random baseline on all four metrics. For $\mathrm{AUCC}_{\mathrm{KEV}}$, the ILP value of 0.997 lies above the 95% CI upper bound of 0.902, providing statistical evidence that the ILP's KEV prioritisation is not achievable by random ordering alone. For #actions, the ILP requires 435 vs. a random mean of 504.1 (95% CI $[494.2, 515.6]$), a reduction of 69.1 actions (13.7% fewer).

The random baseline's $\mathrm{AUCC}_{\mathrm{KEV}}$ mean of 0.704 (CI $[0.579, 0.890]$) is notably wide, reflecting the high variance of random fix ordering. In the best random seeds, lucky KEV-early positioning produces $\mathrm{AUCC}_{\mathrm{KEV}}$ close to 0.89, underscoring that the *worst-case* exposure under random ordering is substantially worse (0.579).

F. Case Study: ExactTarget/fuelux

To illustrate KEVGRAPH's practical output within the corpus-level plan, we examine **ExactTarget/fuelux** (5,152 stars), a JavaScript UI component library with 664 dependency nodes in its lockfile graph.

Vulnerability landscape. The OSV join identified **194 unique vulnerabilities** across packages in this repository's dependency graph. Of these, **3 are KEV-listed**:

- `GHSA-qqvq-6xgj-jw8g` (electron 1.6.11): libvpx heap buffer overflow, CVSS 8.8
- `GHSA-j7hp-h8jx-5ppr` (electron 1.6.11): libwebp OOB write, CVSS 8.8
- `GHSA-jpcq-cgw6-v4j6` (jquery 3.2.1): XSS vulnerability, CVSS 6.9 (KEV added 2025-01-23)

Corpus plan — first actions. The corpus-level KEVGraph plan's first upgrade action is:

`upgrade:electron@1.6.11` → `@35.7.5`

This single action covers **22 vulnerabilities** corpus-wide, including both KEV-listed electron vulnerabilities present in fuelux (CVSS 8.8 each). Within four plan steps all five KEV vulnerabilities across the entire corpus are resolved: electron at step 1, vite at step 2, jquery at step 3, and puppeteer at step 4.

Comparison with Dependabot. Dependabot's alphabetical-within-severity-bucket ordering delays the electron upgrade to step 19 (kev_first_rank $= 19$), leaving the libvpx and libwebp KEV vulnerabilities unpatched through 18 intervening upgrades. An organisation applying the Dependabot ordering expends engineering effort on lower-risk packages before addressing actively exploited ones.

Certificate. The corpus-level remediation certificate records all 912 (fix, vuln) coverage edges, enabling automated verification in under 200 ms that all coverable vulnerabilities—including the three KEV-listed ones present in fuelux—are addressed by the plan.

VII. DISCUSSION

EPSS-first is competitive. The EPSS-first baseline achieves kev_first_rank $= 1$ and $\mathrm{AUCC}_{\mathrm{KEV}} = 0.987$, close to the ILP's 0.997. This suggests that when EPSS scores are available and current, they provide a reasonable proxy for KEV ordering. However, EPSS scores are probabilistic and lag behind KEV list additions by the FIRST API update cadence; KEV membership is a deterministic, authoritative signal.

CVSS is a poor proxy for exploitability. CVSS-first's kev_first_rank $= 18$ and $\mathrm{AUCC}_{\mathrm{KEV}} = 0.639$ demonstrate that high theoretical severity does not predict active exploitation. Several packages with CVSS < 7.0 are KEV-listed while many CVSS 9.0+ vulnerabilities are not. This finding aligns with prior empirical studies of KEV vs. CVSS correlation [15].

Set-cover overlap is essential. Without per-version fix generation, the fix set contains no overlap and every plan degenerates to the same minimum-cover result. The 68.1% overlap rate produced by per-version generation gives the ILP and greedy planners meaningful optimisation choices.

Greedy vs. ILP trade-off. The greedy planner covers more total vulnerabilities in the first 5 steps ($T_5 = 0.091$ vs. 0.061 for ILP) but achieves a lower $\mathrm{AUCC}_{\mathrm{KEV}}$ (0.910 vs. 0.997). Practitioners who need to maximise total coverage in a short window (e.g., a 5-upgrade sprint) should prefer the greedy plan; those who want to minimise KEV exposure above all else should prefer the ILP plan.

Practical relevance to KEV-driven compliance. Under BOD 22-01 and its civilian equivalents, federal agencies and operators of critical infrastructure must remediate KEV-listed vulnerabilities on mandated timelines. The 18–19 step delay introduced by CVSS-first and Dependabot orderings means that

KEV-affected packages—with confirmed active exploitation—are systematically deprioritised relative to theoretically severe but unexploited CVEs. KEVGRAPH's ILP planner eliminates this gap by treating KEV membership as a hard ordering constraint within a formally optimal remediation set. The accompanying certificate provides the machine-readable coverage proof required for automated compliance audit pipelines.

Reproducibility. The full pipeline is deterministic given fixed random seeds and a snapshotted KEV catalogue (`KEVGRAPH_KEV_SNAPSHOT` environment variable). All intermediate artefacts are persisted to disk and all results are regenerable with:

```
python -m src.pipeline --stage plan
```

VIII. Limitations

Corpus size. The current evaluation covers 465 repositories. While the selected repositories span diverse domains and contain over 900 vulnerabilities, generalisability to the broader npm ecosystem requires evaluation at larger scale. We leave scale-up to future work.

KEV sparsity. Only 5 of 936 vulnerabilities (0.53%) are KEV-listed in this corpus. This sparsity means that KEV-first ordering has limited impact on total coverage curves (T_1, T_5) but has a large impact on $\mathrm{AUCC}_{\mathrm{KEV}}$ and kev_first_rank. Corpora with higher KEV density would provide stronger differentiation signals.

Transitive upgrade effects. KEVGRAPH treats each upgrade as independent. In practice, upgrading package A may force a version change in package B (through shared transitive dependencies), which may resolve or introduce additional vulnerabilities. We do not model these transitive effects.

Fix feasibility. KEVGRAPH assumes that upgrading to the latest fixed version is always feasible (no breaking API changes). In practice, major-version upgrades may require code changes that are not captured in the remediation plan.

OSV and KEV freshness. Results depend on the state of the OSV database and the CISA KEV catalogue at the time of the `join` stage. Snapshotting (as done in our reproduction setup) ensures experiment reproducibility but means results do not reflect vulnerabilities discovered after the snapshot date.

IX. Related Work

Dependency vulnerability prioritisation. Decan et al. [13] study the propagation of vulnerabilities through the npm ecosystem. Kikas et al. [14] analyse dependency network structures. Neither work addresses prioritised remediation planning.

Automated dependency updates. Dependabot [5], Renovate [7], and commercial software composition analysis tools provide automated pull requests for dependency upgrades ordered by severity bucket. KEVGRAPH differs by using KEV membership as the primary ordering signal and by minimising total upgrade count via set-cover optimisation—two properties absent from existing tooling.

Vulnerability scoring and exploitation. Spring et al. [15] show that CVSS score is a poor predictor of exploitation. The EPSS system [3] provides daily probability estimates of exploitation within 30 days. Our results confirm the CVSS finding in the npm context—CVSS-first defers KEV fixes to step 18—and further show that even EPSS-first, while competitive, is slightly suboptimal compared to exact ILP optimisation when KEV membership is the target signal.

Formal optimisation for security. Set-cover and integer programming have been applied to network hardening [8]. Our contribution is the application of ILP minimum set cover to the *ordered* remediation problem, with KEV-early as an explicit secondary objective and machine-verifiable certificates as output.

X. Conclusion

We presented KEVGRAPH, an end-to-end pipeline for exploitation-aware, formally optimal vulnerability remediation in npm dependency ecosystems. By framing remediation as a KEV-aware set-cover problem, KEVGRAPH minimises the number of required upgrade actions while ensuring that actively exploited (KEV-listed) vulnerabilities are resolved as early as possible in the remediation sequence—directly supporting KEV-driven compliance mandates such as CISA BOD 22-01.

Evaluated on 465 real-world repositories with 936 vulnerabilities (5 KEV-listed), our ILP planner achieves $\mathrm{AUCC}_{\mathrm{KEV}} = 0.997$ versus a random-baseline mean of 0.688 (bootstrap 95% CI $[0.541, 0.902]$, $n = 30$), ranks the first KEV vulnerability at plan step 1, and requires only 435 upgrade actions (13.7% fewer than the random mean of 504.1). CVSS-first and Dependabot-style ordering—the current industry default—defer the first KEV fix to steps 18 and 19, respectively, leaving actively exploited packages unaddressed through an avoidable remediation window.

The accompanying remediation certificate enables sub-200 ms independent verification that the plan covers all reachable vulnerabilities, supporting automated compliance workflows.

Future work will scale the evaluation to thousands of repositories, model transitive upgrade effects, and integrate KEVGRAPH's output directly into pull-request generation pipelines.

Reproducibility

All code, data artefacts, and figures are available in the project repository. To regenerate all results from pre-cached intermediate artefacts (no network access required):

```
pip install -r requirements.txt
python -m src.pipeline --stage plan
cat data/results.csv          # Table 2
cat data/random_ci.json       # Table 3
```

Expected key values: ILP $\mathrm{AUCC}_{\mathrm{KEV}} = 0.9972$, ILP #actions $= 435$, random $\mathrm{AUCC}_{\mathrm{KEV}}$ mean $= 0.688$, random #actions mean $= 504.1$. To regenerate from raw lockfiles (requires GitHub and OSV API access):

```
python -m src.pipeline --resume
```

All four figures (`data/plots/`) are regenerated automatically at the plot stage.

Acknowledgements

Pipeline code, evaluation artefacts, and reproduction instructions are available at the project repository. All data artefacts required for reproduction are included under `data/`.

References

[1] Cybersecurity and Infrastructure Security Agency (CISA), "Known Exploited Vulnerabilities Catalog," https://www.cisa.gov/known-exploited-vulnerabilities-catalog, Accessed March 2026.

[2] Open Source Vulnerability (OSV) Database, https://osv.dev, Accessed March 2026.

[3] FIRST.org, "Exploit Prediction Scoring System (EPSS)," https://www.first.org/epss/, Accessed March 2026.

[4] FIRST.org, "Common Vulnerability Scoring System v3.1: Specification Document," https://www.first.org/cvss/v3.1/specification-document, 2019.

[5] GitHub, "Dependabot security updates," https://docs.github.com/en/code-security/dependabot, Accessed March 2026.

[6] npm, "npm audit," https://docs.npmjs.com/cli/v10/commands/npm-audit, Accessed March 2026.

[7] Mend.io, "Renovate Bot," https://docs.renovatebot.com, Accessed March 2026.

[8] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[9] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An Analysis of Approximations for Maximizing Submodular Set Functions—I," *Mathematical Programming*, vol. 14, no. 1, pp. 265–294, 1978.

[10] D. S. Johnson, "Approximation Algorithms for Combinatorial Problems," *Journal of Computer and System Sciences*, vol. 9, no. 3, pp. 256–278, 1974.

[11] J. Forrest and R. Lougee-Heimer, "CBC User Guide," in *Emerging Theory, Methods, and Applications*, INFORMS, 2005, pp. 257–277.

[12] S. Mitchell, M. OSullivan, and I. Dunning, "PuLP: A Linear Programming Toolkit for Python," Department of Engineering Science, University of Auckland, 2011.

[13] A. Decan, T. Mens, and E. Constantinou, "On the Impact of Security Vulnerabilities in the npm Package Dependency Network," in *Proc. MSR*, 2018, pp. 181–191.

[14] R. Kikas, G. Gousios, M. Dumas, and D. Pfahl, "Structure and Evolution of Package Dependency Networks," in *Proc. MSR*, 2017, pp. 102–112.

[15] J. Spring et al., "Time to Change the CVSS?," in *Proc. IEEE EuroS&P Workshops*, 2021.